HEAP SORT ALGORITHM PROJECT

NAME	REG NO.
1. **CHARLES KAMAU MAINA**	EB3/67291/23
2. **TYRINE NJERI GITAU**	EB3/67228/23
3. **EVANS LEMAIYAN**	EB3/67270/23
4. **FRANCIS NG'ANGA NJERI**	EB3/67239/23
5. **KELVIN MUNENE**	EB3/67323/23
6. **MARK MOMANYI**	EB3/67324/23
7. **KELVIN KAIRUKI**	EB3/67290/23
8. **PETER KIVWAU**	EB3/67246/23

Heap Sort Complexity Analysis

Heap Sort works in two main phases:

1. Build Max Heap.
2. Extract elements one by one.

A. Best Case

When does it occur?

Even if:

- The array is already sorted (ascending or descending)
- All elements are identical
- Heap Sort still performs the same operations.

Why?

Because:

- It must build the heap structure.
- It must repeatedly swap and heapify.
- Unlike Quick Sort, Heap Sort does not benefit from already sorted data.

Time Complexity

- Build heap → O(n) (O stands for "Order of)
- Extract n elements → O(n log n)

$$T(n) = O(n) + O(n \log n)$$

Best Case = O(n log n)

B. Worst Case

When does it occur?

Worst case happens when:

- Every heapify operation moves the root all the way down the tree.
- Maximum number of swaps per extraction.

Example:

- Strictly increasing order
- Strictly decreasing order

Why?

- Each extraction requires:
- Swapping root with last element.
- Heapifying down to leaf level (height = log n).

Since this happens n times:

$$n \times \log n$$

Worst Case = O(n log n)

C.Average Case

When does it occur?

- For random input data.

Why?

On average:

- Heapify operations traverse roughly log n levels.
- n elements are extracted.

So:

Average Case = O(n log n)

D.Space Complexity

- In-place algorithm
- No extra arrays used

Space Complexity = O(1)

Summary Table

Case	Time Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space	O(1)

HEAP SORT ALGORITHM PROJECT

Code Implementation (C++)

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <chrono>

using namespace std;

// Heapify function
void heapify(vector<int>& arr, int n, int i) {
    while (true) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n && arr[left] > arr[largest])
            largest = left;

        if (right < n && arr[right] > arr[largest])
            largest = right;

        if (largest == i)
            break;

        swap(arr[i], arr[largest]);
        i = largest;
    }
}

// Heap Sort function with timing separation
void heapSort(vector<int>& arr) {
    int n = arr.size();

    auto build_start = chrono::high_resolution_clock::now();

    // Build max heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    auto build_end = chrono::high_resolution_clock::now();

    auto sort_start = chrono::high_resolution_clock::now();

    // Extract elements
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
```

```
    }
    auto sort_end = chrono::high_resolution_clock::now();

    chrono::duration<double> build_time = build_end - build_start;
    chrono::duration<double> sort_time = sort_end - sort_start;

    cout << "Heap Build Time: " << build_time.count() << " seconds\n";
    cout << "Extraction Phase Time: " << sort_time.count() << " seconds\n";
}

int main() {
    const int SIZE = 10000000;  // 10 Million
    vector<int> arr(SIZE);
    std::cout << "Generating " << SIZE << " random integers..." << std::endl;

    // Generate random numbers
    for (int i = 0; i < SIZE; i++)
        arr[i] = rand();

    cout << "Sorting 10,000,000 elements using Heap Sort...\n";

    auto total_start = chrono::high_resolution_clock::now();

    heapSort(arr);

    auto total_end = chrono::high_resolution_clock::now();
    chrono::duration<double> total_time = total_end - total_start;

    cout << "Total Time: " << total_time.count() << " seconds\n";

     // Verification: Check if the array is actually sorted
    bool sorted = true;
    for (int i = 0; i < SIZE - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            sorted = false;
            break;
        }
    }

    if (sorted) {
        std::cout << "Verification successful: The list is sorted." <<
std::endl;
    } else {
        std::cout << "Verification failed: The list is NOT sorted." <<
std::endl;
    }
    return 0;
}
```

Output

```
Generating 10000000 random integers...
Sorting 10,000,000 elements using Heap Sort...
Heap Build Time: 0.140936 seconds
Extraction Phase Time: 3.18552 seconds
Total Time: 3.32719 seconds
Verification successful: The list is sorted.
```